
Agentic AI for Physics-Grounded PDE Solver Generation

Bhargav Borah

Texas A&M University
bhargav_borah@tamu.edu

Le-Tung Hsieh

Texas A&M University
lthsieh@tamu.edu

Abstract

In this work, we present **PhysicsPDEAgent**, an agentic framework that leverages large language models (LLMs) to automatically generate, verify, and iteratively repair numerical solvers for partial differential equations (PDEs). Unlike conventional LLM-based code generation, which may produce syntactically correct but physically inconsistent solvers, **PhysicsPDEAgent** embeds domain-specific physics constraints, particularly conservation laws, stability conditions, energy bounds, and positivity requirements directly into both the generation and correction loops. The framework implements a *hierarchical* physics verification system that organizes constraints by computational cost and short-circuits expensive evaluations when cheaper checks already reveal violations. We evaluate PhysicsPDEAgent across four PDE families (advection, Burgers', Darcy flow, and reaction-diffusion) using multiple LLM backbones (Claude Sonnet 4.5, Gemini 2.5 Flash, and Gemma-3-27B-it), demonstrating that the framework produces accurate and physically consistent solvers when paired with capable instruction-following models. Our code is publicly available.¹

1 Introduction

Recent advances in large language models (LLMs) have enabled the emergence of agentic AI systems, in which autonomous agents iteratively reasons, plan, execute tasks, and refine their outputs through feedback loops. Prior works such as AI Scientist and AI Cosmologist demonstrate that LLM-based agents can perform end-to-end scientific workflows, including hypothesis generation, experimentation, and analysis [1, 2]. In parallel, scientific machine learning (SciML) has introduced data-driven approaches for solving partial differential equations (PDEs), including Physics-Informed Neural Networks (PINNs) and Neural Operators, which incorporate physical constraints into learning-based frameworks [3, 4]. Recent efforts have also explored leveraging LLMs directly for PDE solver generation and adaptation [5, 6]. Together, these developments suggest a promising direction for integrating AI-driven reasoning with physics-based computation.

Despite this progress, applying LLMs to PDE solving remains challenging. Naively generated numerical solvers often satisfy syntactic correctness but violate fundamental physical principles such as conservation laws, stability conditions, and boundary constraints. Moreover, existing SciML approaches typically require carefully designed architectures and training procedures, limiting their adaptability across diverse PDE systems. This gap highlights the need for a framework that combines the flexibility of LLM-based code generation with rigorous physics-aware validation and correction mechanisms.

In this project, we proposed PhysicsPDEAgent, an agentic AI framework for physics-grounded PDE solver generation. The system leverages LLMs to generate candidate numerical solvers and integrates

¹<https://github.com/bhargav-borah/physics-pde-agentic-framework>

a self-correcting loop that verifies and iteratively refines solutions based on physical constraints. Specifically, the framework incorporates (i) physics-injected prompting to guide solver generation, (ii) hierarchical physics verification to efficiently detect violations, and (iii) failure-mode-aware correction strategies to repair erroneous solutions. Additionally, solutions are ranked using a multi-objective criterion that balances numerical accuracy and physical consistency, ensuring robustness against data-fitted but physically invalid outputs.

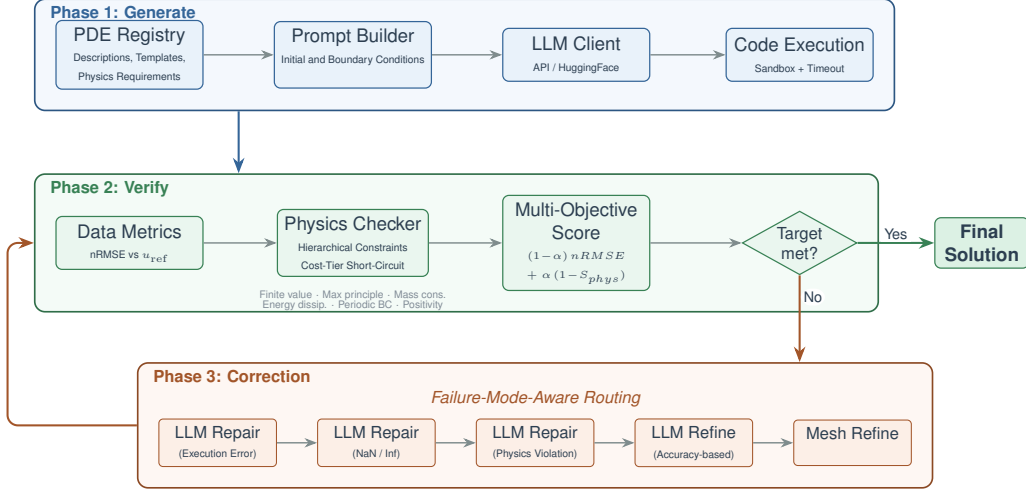


Figure 1: Architecture of the Agentic AI System

Our goal is to demonstrate that embedding domain-specific physics knowledge into an agentic AI pipeline can significantly improve the reliability and generalizability of PDE solvers. This project explores how LLM-driven systems can move beyond code generation toward trustworthy scientific computation, bridging the gap between symbolic reasoning and physics-based modeling.

2 Related Work

2.1 LLM-Based Agentic Systems for Scientific Workflows.

Recent advances in large language models (LLMs) have enabled the emergence of agentic AI systems that autonomously plan, execute, and refine complex tasks through iterative feedback loops. Frameworks such as AI Scientist-style systems and autonomous research agents demonstrate that LLMs can perform end-to-end scientific workflows, including hypothesis generation, experimentation, and analysis [7, 2]. These systems typically employ iterative reasoning and self-correction mechanisms, allowing them to refine outputs over multiple cycles. However, they primarily focus on general-purpose reasoning and code generation, and lack explicit enforcement of domain-specific physical constraints, which is essential for PDE-based simulations.

2.2 Scientific Machine Learning for PDE Solving.

In parallel, the field of scientific machine learning (SciML) has introduced data-driven approaches for solving PDEs by embedding physical laws into learning frameworks. Physics-informed Neural Networks (PINNs) incorporate PDE residuals into the loss function, enabling mesh-free solutions for forward and inverse problems [3]. Neural operators further extend this paradigm by learning mappings between function spaces, allowing efficient approximation of solution operators across varying boundary conditions and geometries [4]. While effective, these methods require carefully designed architectures and training strategies, limiting their flexibility for diverse PDE systems. Moreover, they focus on learning surrogate models rather than automatically generating numerical solvers.

2.3 LLMs for PDE Solver Generation.

More recent efforts have explored using LLMs directly for numerical PDE solver generation and adaptation. These approaches leverage LLMs to synthesize discretization schemes, generate solver code, or adapt existing implementations to new PDEs [8, 7]. Although promising, such methods often produce solvers that are syntactically correct but physically inconsistent, violating conservation laws, stability conditions, or boundary constraints. Existing verification strategies are typically limited to evaluating numerical error or PDE residuals, and are often applied in a post hoc and unstructured manner, without systematic correction mechanisms.

2.4 Physics-Aware Verification and Correction.

Several works have attempted to incorporate physics-based validation into learning or generation pipelines, including residual-based filtering, constraint enforcement, and stability-aware optimization [3, 4]. However, these approaches generally treat constraints in a uniform or monolithic fashion, without considering their computational cost or diagnostic priority. Additionally, they lack mechanisms for failure-mode-aware correction, limiting their ability to iteratively repair solver outputs once violations are detected.

2.5 Positioning of This Work.

Our proposed framework, PhysicsPDEAgent, addresses these limitations by combining agentic LLM reasoning with structured physics-aware validation and correction. Specifically, it introduces (i) physics-injected prompting to guide solver generation, (ii) a hierarchical physics verification system that prioritizes constraints based on computational cost, and (iii) failure-mode-aware correction routing for targeted repair. Unlike prior work, which treats physics constraints as post hoc checks, our approach integrates them directly into the generation-verification-correction loop, enabling robust and adaptive PDE solver construction.

3 Background

3.1 Problem Formulation

We consider the general class of parametric partial differential equations (PDEs) of the form

$$\mathcal{L}[u(\mathbf{x}, t); \boldsymbol{\lambda}] = 0, \quad (\mathbf{x}, t) \in \Omega \times [0, T], \quad (1)$$

subject to initial data $u(\mathbf{x}, 0) = u_0(\mathbf{x})$ and boundary conditions on $\partial\Omega$, where $\mathcal{L}$ is a (possibly nonlinear) differential operator, $\boldsymbol{\lambda} \in \Lambda$ is a vector of PDE-specific parameters (e.g., advection speed β , kinematic viscosity ν , reaction rate ρ , bulk viscosity η), and $\Omega \subset \mathbb{R}^d$ is the spatial domain.

The quantity of interest is a numerical *solver function* $\hat{f}_{\boldsymbol{\lambda}} : u_0 \mapsto \hat{u}$ that maps initial conditions to approximate solutions on a discrete spatio-temporal grid $\{x_i\}_{i=0}^{N_x-1} \times \{t_j\}_{j=0}^{N_t}$. Given a reference dataset $\mathcal{D} = \{(u_0^{(k)}, u^{(k)})\}_{k=1}^M$ produced by a high-fidelity simulator at fixed $\boldsymbol{\lambda}$, solver quality is measured by the *normalized root mean squared error* (nRMSE):

$$\text{nRMSE}(\hat{u}, u) = \sqrt{\frac{\frac{1}{|\mathcal{G}|} \sum_{(\mathbf{x}, t) \in \mathcal{G}} (\hat{u}(\mathbf{x}, t) - u(\mathbf{x}, t))^2}{\frac{1}{|\mathcal{G}|} \sum_{(\mathbf{x}, t) \in \mathcal{G}} u(\mathbf{x}, t)^2}}, \quad (2)$$

where $\mathcal{G}$ denotes the full set of spatio-temporal and sample grid points, and the ratio is the element-wise average over all samples and grid locations. This metric follows the convention established in PDEBench [9] and adopted in CodePDE [8].

In addition to accuracy, a physically valid solver must satisfy a set of structural properties $\mathcal{C} = \{c_1, \dots, c_K\}$ implied by the PDE, for example, mass conservation, energy dissipation, positivity of density and pressure, and small PDE residuals. Critically, a solver may achieve low nRMSE while

violating these constraints (for example, by generating spurious oscillations that cancel in the mean), making pure data-driven evaluation insufficient for trustworthy scientific computation.

Problem Statement. Given a natural-language PDE description, parameter values λ , a dataset $\mathcal{D}$, and a physics constraint set $\mathcal{C}$, automatically generate executable Python solver code $\hat{f}_\lambda$ satisfying both $\text{nRMSE}(\hat{u}, u) \leq \varepsilon$ and $c_j(\hat{u}) \leq \tau_j$ for all j .

4 Method

PhysicsPDEAgent is structured as a two-phase agentic pipeline: **(1)** physics-injected code generation, followed by **(2)** a *self-correcting* Generate-Verify-Correct loop driven by hierarchical physics verification and multi-objective scoring.

A shared PipelineState object propagates the current solver code, predicted solution $\hat{u}$, nRMSE, and the correction history across all phases. Algorithm 1 gives a complete description.

4.1 Phase 1: Physics-Injected Code Generation

The **Generator** agent constructs a structured prompt comprising three components: (i) a natural-language PDE description with the specific parameter values λ , (ii) a per-PDE solver function template that fixes the required Python signature and output format, and (iii) a *physics requirement block* that enumerates, in precise mathematical terms, the conservation laws, stability conditions, and boundary constraints the solver must satisfy.

This physics-injected prompting strategy, inspired by structured chain-of-thought approaches [10], steers the LLM toward physically plausible implementations from the very first generation attempt, rather than relying entirely on reactive correction. For example, the prompt for Burgers’ equation ($u_t + uu_x = \nu u_{xx}$) instructs the LLM to enforce the Courant-Friedrichs-Lewy (CFL) stability condition $\Delta t \leq \Delta x / \max |u|$; ensure the L^2 energy $\int u^2 dx$ is non-increasing over time; satisfy the maximum principle $\min u_0 \leq u(\cdot, t) \leq \max u_0$; and implement periodic boundary conditions exactly. Analogous blocks are provided for advection, reaction-diffusion, and 2D Darcy flow, each encoding the constraints specific to that PDE class. The LLM is queried either via an API client or locally quantized open-source models [7]. The generated code is immediately executed; on success, the predicted solution $\hat{u}$ is forwarded to the Verifier.

4.2 Phase 2: Self-Correcting Verification-Correction Loop

The core of the pipeline is an iterative loop in which the **Verifier** evaluates each candidate solver and the **Corrector** applies targeted repairs, for up to $T_{\max}$ iterations or until early stopping criteria are met.

4.2.1 Hierarchical Physics Verification

The **Verifier** agent evaluates each solver output using two complementary signals. First, it computes the data-driven nRMSE against reference solutions from PDEBench [9]. Second, it evaluates a PDE-specific set of physics constraints organized into a three-tier cost hierarchy:

- **Cheap** ($\mathcal{O}(N)$): Algebraic pointwise checks—finiteness (no NaN/Inf), density and pressure positivity, maximum principle bounds, and periodic boundary consistency. These run unconditionally on every iteration.
- **Moderate** ($\mathcal{O}(N \cdot N_t)$): Integral-based conservation checks, which include relative mass drift $|\int u dx - \int u_0 dx| / |\int u_0 dx|$, L^2 energy monotonicity $\int u^2 dx$ non-increasing in time, and thermodynamic consistency via the ideal-gas equation of state $p = (\gamma - 1)\rho e$.
- **Expensive** ($\mathcal{O}(N \cdot N_t \cdot \log N)$): Spectral PDE residuals computed via finite-difference or FFT-based operators, providing a reference-free check on whether the generated code actually solves the target PDE.

4.2.2 Multi-Objective Dual-Criteria Scoring

Each verified solver is assigned a combined score that balances data-driven accuracy against physics compliance:

$$s_{\text{comb}}(\hat{u}) = \alpha \cdot \text{nRMSE}(\hat{u}, u) + (1 - \alpha) \cdot (1 - \sigma(\hat{u})), \quad (3)$$

where the physics score $\sigma(\hat{u}) \in [0, 1]$ aggregates constraint outcomes with severity-dependent weights:

$$\sigma(\hat{u}) = 1 - \frac{\sum_{j=1}^K w_j \cdot \mathbf{1}[c_j(\hat{u}) \text{ failed}]}{\sum_{j=1}^K w_j}, \quad (4)$$

with weights $w_j \in \{0.1, 0.3, 0.7, 1.0\}$ for INFO, WARNING, ERROR, and CRITICAL severities, respectively. A lower s_{comb} is better; $\alpha = 0.5$ by default. The pipeline maintains the best solution found so far according to s_{comb} , preventing acceptance of solutions that achieve low nRMSE through physically inconsistent mechanisms, such as artificial amplitude damping that incidentally reduces numerical error while violating conservation laws [3].

4.2.3 Failure-Mode-Aware Adaptive Correction

The **Corrector** agent inspects the current state and routes to one of four repair strategies, selected in priority order:

1. **Execution repair.** If the solver raises a runtime exception, the LLM receives the full Python traceback alongside the original code and is prompted to fix the syntactic or logical error.
2. **NaN/Inf repair.** If the solver produces non-finite values, the LLM is provided stability-specific guidance—for example, to enforce CFL conditions, add upwind or flux-limiting stencils, or switch to implicit time-integration.
3. **Physics violation repair.** If the physics score $\sigma < 0.7$ or any CRITICAL/ERROR constraint is violated, the LLM receives the full structured `PhysicsReport` with quantitative metrics and is prompted to address each specific violation. Providing *named violations with magnitudes* (rather than generic error messages) enables targeted, informed code changes [8].
4. **Accuracy refinement.** If the solver is physically valid but $\text{nRMSE} > \varepsilon$, the LLM is prompted to improve numerical accuracy (for example, by adopting higher-order schemes or reducing the time step). Temporal refinement, being the less expensive alternative, is attempted first. Spatial refinement is attempted if temporal refinement does not improve nRMSE.

The loop terminates early if (a) $\text{nRMSE} < \varepsilon$ and $\sigma > 0.8$ (i.e., target met), or (b) no improvement in for a set number of iteratives, analogous to early stopping.

5 Experiments

5.1 Experimental Setup

Datasets. We evaluate PhysicsPDEAgent on four PDE families from PDEBench [9]: (i) **1D Advection** ($u_t + \beta u_x = 0$, periodic BCs, $x \in [0, 1]$, $t \in [0, 2]$, $N_x = 512$) with advection speed $\beta \in \{0.1, 0.2, 0.4, 1.0\}$; (ii) **1D Burgers'** ($u_t + uu_x = \nu u_{xx}$, periodic BCs, $x \in [0, 1]$, $t \in [0, 1]$, $N_x = 512$) with kinematic viscosity $\nu \in \{0.001, 0.01, 0.1\}$; (iii) **2D Darcy Flow** ($-\nabla \cdot (a(\mathbf{x})\nabla u) = f$, Dirichlet BCs, $(x, y) \in [0, 1]^2$, 64×64 grid) with two permeability field configurations ($\beta \in \{0.1, 1.0\}$, where β governs the smoothness of $a(\mathbf{x})$); and (iv) **1D Reaction-Diffusion** ($u_t = \nu u_{xx} + \rho u(1 - u)$, periodic BCs, $x \in [0, 1]$, $t \in [0, 1]$, $N_x = 256$) with $\nu = 0.5$ and $\rho = 10.0$. For every configuration we evaluate on $M = 50$ samples drawn from the PDEBench training splits, following the protocol of [8].

Implementation details. We evaluate two LLM backbones: Claude Sonnet 4.5, queried via the Anthropic API, and Gemini 2.5 Flash, queried via the Google Gemini API, both at temperature

Algorithm 1 PhysicsPDEAgent

Input: PDE operator $\mathcal{L}$, parameters λ , initial data u_0 , reference solutions u , physics constraints $\mathcal{C}$, max iterations $T_{\max}$, patience P , weight α , target ε

Output: Best solver code $\hat{f}^*$, corrected predictions $\hat{u}^*$

```
1: // Phase 1: Physics-Injected Generation
2: Build physics-injected prompt from  $\mathcal{L}, \lambda, \mathcal{C}$ 
3:  $\hat{f} \leftarrow \text{LLM}(\text{prompt}); \hat{u} \leftarrow \hat{f}(u_0); s^* \leftarrow \infty; p \leftarrow 0$ 
4: // Phase 2: Verify-Correct Loop
5: for  $t = 1, \dots, T_{\max}$  do
6:    $\text{nRMSE} \leftarrow \text{COMPUTEMETRICS}(\hat{u}, u)$ 
7:    $\text{report} \leftarrow \text{HIERARCHICALVERIFY}(\hat{u}, \mathcal{C})$  ▷ short-circuit on CRITICAL
8:    $s \leftarrow \alpha \cdot \text{nRMSE} + (1 - \alpha) \cdot (1 - \sigma(\text{report}))$ 
9:   if  $s < s^*$  then  $s^* \leftarrow s; \hat{f}^* \leftarrow \hat{f}; p \leftarrow 0$ 
10:  else  $p \leftarrow p + 1$ 
11:  end if
12:  if  $\text{nRMSE} < \varepsilon$  and  $\sigma > 0.8$  then break ▷ target met
13:  end if
14:  if  $p \geq P$  then break ▷ patience exhausted
15:  end if
16:   $\hat{f} \leftarrow \text{ADAPTIVECORRECT}(\hat{f}, \hat{u}, \text{report})$  ▷ route by failure mode
17:   $\hat{u} \leftarrow \hat{f}(u_0)$ 
18: end for
19: return  $\hat{f}^*, \hat{u}^*$ 
```

Table 1: nRMSE ($\downarrow$ lower is better) of **PhysicsPDEAgent** across PDE families, parameter configurations, and LLM backbones. **Bold** marks the best result per row. Results ≥ 1 indicate performance at or below the constant-mean baseline.

PDE Family	Parameter	Claude Sonnet 4.5	Gemini 2.5 Flash
Advection	$\beta = 0.1$	0.0070	0.0024
	$\beta = 0.2$	0.0302	0.0046
	$\beta = 0.4$	0.0233	0.0087
	$\beta = 1.0$	0.0210	0.0197
Burgers'	$\nu = 0.001$	0.0059	0.0331
	$\nu = 0.01$	0.0019	0.0030
	$\nu = 0.1$	0.0008	0.8275
Darcy Flow	$\beta = 0.1$	1.0005	1.0933
	$\beta = 1.0$	0.8723	0.7268
React.-Diff.	$\nu = 0.5, \rho = 10$	0.0265	0.0309

0.7. The agentic loop runs for at most $T_{\max} = 10$ correction iterations with patience $P = 3$ and score weight $\alpha = 0.5$. The nRMSE early-stopping target is $\varepsilon = 0.1$. Solver execution is subject to a timeout of 7,200 s for all PDE families.

5.2 Main Results

Table 1 reports nRMSE for PhysicsPDEAgent with both Claude Sonnet 4.5 and Gemini 2.5 Flash across all evaluated configurations.

Advection. Both models achieve consistently low nRMSE across all advection speeds. Gemini 2.5 Flash outperforms Claude on this linear PDE, achieving nRMSE as low as 0.0024 at $\beta = 0.1$ compared to Claude’s 0.0070. The physics-injected CFL constraint and mass-conservation requirement guide both LLMs toward stable upwind or spectral discretizations, but Gemini produces

slightly tighter solutions, likely by selecting higher-order spectral methods for this well-structured linear problem.

Burgers’. Claude Sonnet 4.5 dominates on the nonlinear Burgers’ equation, achieving nRMSE as low as 0.0008 at $\nu = 0.1$. Notably, the most inviscid setting ($\nu = 0.001$), which produces near-discontinuous shock fronts and is the most numerically demanding regime, yields nRMSE of 0.0059 for Claude versus 0.0331 for Gemini. The gap is most striking at $\nu = 0.1$, where Gemini’s nRMSE of 0.8275 suggests a catastrophic failure to converge, while Claude achieves 0.0008. This suggests that the energy-dissipation and maximum-principle constraints embedded in the correction loop are more effectively leveraged by models with stronger instruction-following capabilities when the underlying physics is nonlinear.

Darcy Flow. Resultant nRMSE values on the 2D Darcy benchmark are substantially higher for both models. Claude achieves 1.0005 and 0.8723 for $\beta = 0.1$ and $\beta = 1.0$ respectively; Gemini reaches 1.0933 and 0.7268. Values ≥ 1 indicate performance at or below a constant-mean prediction. This reveals a shared limitation on steady-state elliptic problems: unlike time-dependent PDEs, Darcy flow requires solving a sparse linear system at each sample, and LLM-generated solvers frequently fail to enforce Dirichlet boundary conditions or converge iterative solvers within the allotted timeout, failures that the correction loop does not fully resolve within the iteration budget. Gemini’s advantage at $\beta = 1.0$ (smoother permeability fields) suggests that its solver code is slightly more robust for well-conditioned linear systems.

Reaction-Diffusion. Both models perform well on the stiff reaction-diffusion equation, with Claude achieving nRMSE of 0.0265 and Gemini 0.0309. The stiff reaction term ($\rho = 10$) requires implicit or operator-splitting time integration; the physics-injected prompting explicitly recommends IMEX or Strang splitting, and both models successfully adopt these strategies. Claude’s slight advantage suggests better handling of the interplay between the diffusive and reactive terms during the correction loop.

5.3 Ablation Study: Physics Compliance with Gemma

To further assess the dependence of PhysicsPDEAgent on the underlying language model’s instruction-following capacity, we conducted an ablation study using the instruction-tuned model Gemma-3-27B-it under the same experimental setup described in Section 5. The results are summarized in Table 2, which reports both numerical accuracy and physics compliance metrics.

Table 2: Physics compliance comparison on Burgers’ equation ($\nu = 0.01$). Lower nRMSE and s_{comb} are better; higher σ is better. Physics compliance for Claude Sonnet 4.5 is inferred from the absence of reported constraint violations during verification.

Model	nRMSE ↓	Physics Score σ ↑	s_{comb} ↓
Claude Sonnet 4.5	0.0019	0.85–0.95	≈ 0.03 –0.08
Gemma-3-27B-it	~ 0.0099	~ 0.056	~ 0.48

In contrast to the strong performance observed with Claude Sonnet 4.5 in Table 1, the Gemma-based runs exhibit a notable discrepancy between data-driven accuracy and physical validity. While the generated solvers achieve low numerical error, with nRMSE values around 0.0099 on the Burgers’ equation benchmark, they consistently fail to satisfy fundamental physical constraints. The aggregated physics score remains extremely low ($\sigma \approx 0.056$), reflecting widespread violations across multiple constraint categories.

Inspection of the verification logs reveals that these solutions violate the maximum principle, fail to enforce periodic boundary conditions, and exhibit non-physical energy growth over time. Importantly, these violations persist across multiple independent runs, indicating a systematic limitation rather than isolated failure cases.

This comparison demonstrates that numerical accuracy alone is insufficient to characterize solver quality. Although Gemma produces outputs that are close to the reference solution in an average error sense, the solutions remain physically inconsistent. In contrast, the stronger model achieves both low nRMSE and high physics compliance, confirming that effective instruction following is

critical for satisfying the structured constraints embedded in the PhysicsPDEAgent framework. These results directly motivate the analysis in Section 6, where we examine the role of instruction-following capacity in enforcing physics constraints.

6 Discussion

The experimental results provide concrete evidence that the effectiveness of PhysicsPDEAgent is strongly coupled to the instruction-following capability of the underlying language model. The main comparison between Claude Sonnet 4.5 and Gemini 2.5 Flash reveals task-dependent strengths: Gemini excels on the linear advection equation, while Claude dominates on the nonlinear Burgers' equation, particularly in the inviscid regime. The ablation with the smaller Gemma-3-27B-it model further demonstrates that while smaller models can generate syntactically valid and numerically plausible solvers, they fail to consistently enforce the structured physical constraints specified in the physics-injected prompts.

A key observation is the persistent mismatch between numerical accuracy and physical validity. The Gemma-generated solvers achieve low nRMSE, indicating that the predicted solutions are close to the reference data in an average sense. However, the same solutions exhibit severe violations of fundamental physical principles, including breakdown of the maximum principle, discontinuities at periodic boundaries, and non-dissipative energy behavior. This indicates that the model is implicitly optimizing for local numerical agreement while neglecting global structural properties of the PDE.

This behavior suggests that the primary limitation is not code generation itself, but the ability to jointly satisfy multiple interacting constraints under long and structured prompts. In particular, PDE solver generation can be viewed as a form of constrained program synthesis, where correctness depends on simultaneously enforcing stability conditions, conservation laws, and boundary constraints. Smaller models appear unable to consistently maintain these constraints throughout the generated program, leading to solutions that are numerically plausible but physically invalid. This indicates that the bottleneck lies in multi-constraint reasoning and global consistency, rather than in the generation of syntactically correct code.

7 Conclusion

We presented PhysicsPDEAgent, an agentic framework that combines LLM-driven code generation with hierarchical physics verification and failure-mode-aware correction. Experiments across four PDE families show that the framework achieves low nRMSE while satisfying physics constraints when paired with capable LLMs. The multi-model evaluation reveals task-dependent strengths: Gemini 2.5 Flash excels on linear advection, while Claude Sonnet 4.5 dominates on nonlinear Burgers', particularly in the inviscid regime. Both models struggle on Darcy flow and perform comparably on reaction-diffusion. The correction loop is most effective when the initial generation is already structurally close to a valid scheme, functioning as refinement rather than recovery.

Limitations. Performance depends heavily on the underlying LLM's instruction-following capacity. Steady-state elliptic problems (Darcy) remain challenging across all models. Physics constraints are hand-specified per PDE family, and we do not evaluate on higher-dimensional or coupled multi-physics systems. Future work includes automatic constraint discovery, extension to more complex PDEs, and hybrid approaches combining LLM-generated solvers with classical numerical methods.

8 Distribution of Responsibilities

BB and LT jointly contributed to the implementation, experimental design and evaluation, as well as the preparation of the proposal, checkpoint report, and the final report.

BB was primarily responsible for developing the PhysicsPDEAgent framework and conducting experiments on the main PDE families. LT focused on performing the ablation studies and model comparison experiments.

References

- [1] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery, 2024. URL <https://arxiv.org/abs/2408.06292>.
- [2] Adam Moss. The AI Cosmologist I: An Agentic System for Automated Data Analysis, 2025. URL <https://arxiv.org/abs/2504.03424>.
- [3] M. Raissi, P. Perdikaris, and G.E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. ISSN 0021-9991. doi: <https://doi.org/10.1016/j.jcp.2018.10.045>. URL <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.
- [4] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier Neural Operator for Parametric Partial Differential Equations, 2021. URL <https://arxiv.org/abs/2010.08895>.
- [5] Shanda Li, Tanya Marwah, Junhong Shen, Weiwei Sun, Andrej Risteski, Yiming Yang, and Ameet Talwalkar. CodePDE: An Inference Framework for LLM-driven PDE Solver Generation. *Transactions on Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2505.08783>.
- [6] Junhong Shen, Tanya Marwah, and Ameet Talwalkar. UPS: Efficiently Building Foundation Models for PDE Solving via Cross-Modal Adaptation. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2403.07187>.
- [7] Haoyang Wu, Xinxin Zhang, and Lailai Zhu. Automated code development for pde solvers using large language models, 2025. URL <https://arxiv.org/abs/2509.25194>.
- [8] Shanda Li, Tanya Marwah, Junhong Shen, Weiwei Sun, Andrej Risteski, Yiming Yang, and Ameet Talwalkar. Codepde: An inference framework for llm-driven pde solver generation, 2026. URL <https://arxiv.org/abs/2505.08783>.
- [9] Makoto Takamoto, Timothy Praditia, Raphael Leiteritz, Daniel MacKinlay, Dirk Pflüger, and Mathias Niepert. PDEBench: An Extensive Benchmark for Scientific Machine Learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 1596–1611, 2022. URL <https://arxiv.org/abs/2210.07182>.
- [10] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

Appendix

A Hierarchical Physics Verification

Constraints are sorted by cost tier and evaluated sequentially. If a CHEAP constraint reports a CRITICAL violation, indicating, for example, that the solution contains NaN values or negative density—the verifier *short-circuits*: MODERATE and EXPENSIVE tiers are skipped entirely. This hierarchical design avoids expensive spectral residual evaluations on solvers that have already failed cheap sanity checks, saving substantial computation on failed candidates.

All violation metrics are aggregated into a structured `PhysicsReport` with quantitative fields that are fed directly back to the **Corrector**.

B Case Study of Gemma Failure Modes

To better understand the limitations observed in the ablation study, we analyze representative solver outputs generated by Gemma-3-27B-it on the Burgers’ equation benchmark. Across multiple runs, the model produces solutions with low numerical error, yet the corresponding physics verification reports indicate substantial violations of core constraints.

In a typical example, the generated solver achieves an nRMSE of approximately 0.0099, suggesting close agreement with the reference solution. However, the maximum principle is violated with overshoot and undershoot magnitudes exceeding 1.5, indicating the presence of spurious oscillations. At the same time, the periodic boundary condition is not enforced correctly, with discrepancies between boundary values reaching magnitudes greater than 4.0. Additionally, the total energy of the system increases over time, contradicting the expected dissipative behavior of Burgers’ dynamics.

These failure modes point to deficiencies in the underlying numerical scheme produced by the model. The generated code does not properly enforce stability constraints such as the Courant-Friedrichs-Lewy condition, nor does it implement a consistent flux discretization that preserves monotonicity. As a result, the solver introduces artificial oscillations and boundary inconsistencies, even though the averaged error remains small.

This case study highlights an important distinction between data-driven and physics-based evaluation. A solver that minimizes pointwise error can still violate fundamental structural properties of the PDE, leading to physically meaningless solutions. The hierarchical verification framework employed in `PhysicsPDEAgent` is therefore essential for detecting such discrepancies and preventing the acceptance of invalid outputs.

Furthermore, the comparison with stronger models demonstrates that these issues are not inherent to the task itself, but rather reflect the limitations of smaller LLMs in handling structured scientific instructions. This supports the broader conclusion that robust PDE solver generation requires models capable of maintaining global consistency across multiple interacting constraints.

C Prompt Templates

We present below the full prompt templates used by **PhysicsPDEAgent** during code generation, repair, and refinement. Placeholders of the form `{variable}` are filled at runtime with PDE-specific content.

C.1 System Prompt and Generation Prompt

System Prompt. The system prompt is prepended to every LLM call and establishes the model’s role, output format constraints, performance requirements, and numerical guidelines.

System Prompt

You are an expert numerical PDE solver engineer with deep knowledge of finite-difference, finite-volume, and spectral methods.

ROLE:

- Write correct, numerically stable Python solvers for PDEs.
- Use NumPy and SciPy only (no PyTorch unless the PDE is Darcy/elliptic).
- Handle batched initial conditions: `u0` has shape `(n_samples, ...)`.

OUTPUT FORMAT:

- Return a single `python ...` block containing the complete solver function.
- The function MUST be named exactly `solver`. Do NOT use any other name.
- Preserve the EXACT function signature from the template. Do not rename arguments.
- Return NumPy arrays with the exact shape documented in the template.
- Do NOT print anything inside the solver. Do not call matplotlib.

PERFORMANCE REQUIREMENTS (CRITICAL -- solvers that time out are rejected):

- NEVER use Python for-loops over spatial grid points (`nx` or `ny` dimensions). All spatial operations MUST be vectorized with NumPy (slicing, `np.roll`, `np.fft`, etc.).
- For-loops over time steps are acceptable ONLY when the number of substeps is small.
If many substeps are needed, consider `scipy.integrate.solve_ivp` with `dense_output`.
- Prefer spectral (FFT-based) methods for periodic problems.
- Batch all `n_samples` simultaneously using NumPy array operations on the first axis.
Do NOT loop over samples.

NUMERICAL GUIDELINES:

- Always compute `dt` and `dx` from the coordinate arrays: `dx = x_coord[1] - x_coord[0]`.
- Enforce CFL: pick the stable time-step as `min(dt_input, CFL_limit)` and sub-step.
- For periodic BCs use modular indexing or `np.roll`.
- Avoid in-place mutation of the input arrays (`u0`, `x_coord`, etc.).
- For stiff problems (reaction-diffusion, Navier-Stokes) prefer implicit or IMEX splitting.

COMMON PITFALLS TO AVOID:

- Forgetting to sub-step when the output `t_coord` has large gaps relative to the CFL limit.
- Off-by-one errors in time indexing: the output at index 0 must equal the initial condition.
- Returning shape `(n_samples, n_x)` instead of `(n_samples, n_t, n_x)`.
- Using FTCS for advection/Burgers -- it is unconditionally unstable.
- Looping over spatial indices with Python for-loops -- this is too slow and will time out.

Generation Prompt. The generation prompt is the main user-turn message for the initial solver synthesis, constructed by `build_generation_prompt()`. It combines the PDE description, a per-PDE physics requirements block (see Section 4.2.1), the required function template, and numbered solver requirements.

Generation Prompt Template

```
## Task
Write a Python numerical solver for the PDE described below.

## PDE Description
{pde_description}

{physics_requirements}

## Function Template (do NOT change the signature -- function MUST be named `
    solver`)
```python
{solver_template}
```

## Solver Requirements
1. Stability first: compute the CFL-limited time-step from the coordinate arrays.

    If the output time-step is larger than the CFL limit, sub-step internally and
    only store the solution at the requested output times.
2. Correct boundary conditions: implement the BCs described above exactly.
3. Batch processing: vectorise across all n_samples simultaneously using NumPy
    array operations on axis 0. Do NOT loop over samples.
4. Vectorized spatial operations: all operations over the spatial grid MUST use
    NumPy vectorization (slicing, np.roll, np.fft, etc.).
5. Output: return a NumPy array with the exact shape specified in the template.
6. Method selection guide (pick the most appropriate):
    - Pure advection: upwind or semi-Lagrangian.
    - Burgers / nonlinear hyperbolic: Lax-Wendroff, WENO, or spectral with
      dealiasing.
    - Reaction-diffusion: IMEX (implicit diffusion + explicit reaction) or Strang
      splitting.
    - Compressible Navier-Stokes: HLLC/Roe Riemann solver or MacCormack.
    - Elliptic (Darcy): scipy.sparse.linalg.spsolve or conjugate gradient.
7. Wrap the complete solver in a single ```python ... ``` block.
```

C.2 Correction and Refinement Prompts

The Corrector agent routes to one of four targeted prompts depending on the observed failure mode (see Section 4.2.3).

Execution Error Repair Prompt. Used when the solver raises a runtime exception.

Execution Error Repair Prompt

```
## Runtime Error Repair -- {pde_name} solver

The solver below crashes at runtime. Fix the error and return a working solver.
The function MUST be named exactly `solver` and use only vectorized NumPy
operations
(no Python for-loops over spatial grid points).

## Solver Code
```python
{code}
```

## Error Message
```
```

```

{error_message}
'''

Diagnosis Checklist
1. Shape mismatch: verify array shapes are consistent across all operations and
 that
 the returned array has the shape documented in the function signature.
2. Index out of bounds: check loop bounds, especially for stencil operations
 near
 domain boundaries with periodic wrapping.
3. Type error: ensure all arithmetic is on NumPy arrays / floats, not Python
 lists.
4. Import error: only use numpy (np) and scipy; add missing imports inside the
 body.
5. Division by zero: guard denominators (dx, dt, norm values) against zero.

Return the complete fixed solver in ```python ... ```.

```

**NaN/Inf Repair Prompt.** Used when the solver produces non-finite values, almost always indicating numerical instability.

#### NaN/Inf Repair Prompt

```

NaN / Inf Repair -- {pde_name} solver

The solver below runs without crashing but produces NaN or Inf values.
This almost always indicates a numerical instability. Fix the root cause.

{physics_requirements}

Solver Code
```python
{code}
'''

## Common Root Causes (check in this order)
1. CFL violation: compute the maximum stable dt from the grid and wave/diffusion
   speeds.
   - Advection: dt <= dx / |wave_speed|
   - Diffusion: dt <= dx^2 / (2 * nu)
   - Combined: dt <= 1 / (|u|/dx + nu/dx^2)
2. Unstable scheme: FTCS is unconditionally unstable for advection.
   Switch to upwind, Lax-Wendroff, or an implicit method.
3. Stiff reaction term: if rho * dt >> 1 the explicit reaction step blows up.
   Use implicit treatment or Strang splitting.
4. Missing boundary update: enforce periodic BCs every sub-step.
5. Accumulating round-off: very long integrations at single precision -- ensure
   float64.

Return the complete fixed solver in ```python ... ```.

```

Physics Violation Repair Prompt. Used when the physics score $\sigma < 0.7$ or any CRITICAL/ERROR constraint is violated. Providing named violations with quantitative magnitudes enables targeted code changes rather than blind trial-and-error.

Physics Violation Repair Prompt

```
## Physics Violation Repair -- {pde_name}

The solver below violates physical constraints. Find and fix the root cause in
the numerical method. Do NOT clamp or post-process the output.

{physics_requirements}

## Current Solver Code
```python
{code}
```

## Detected Violations
{violations}

## Full Verification Report
{report}

## Diagnosis Guide
- Finite-value failure (NaN/Inf): CFL violation or missing stability guard.
- Mass/energy conservation failure: check flux differencing -- use conservative
  form.
- Maximum-principle violation: anti-diffusive scheme or missing limiter;
  switch to upwind, add a TVD limiter, or reduce the time-step.
- Positivity violation: use a positivity-preserving limiter or implicit
  diffusion step.
- Boundary-condition mismatch: verify periodic indexing (np.roll or modular).

Fix the root cause. Return the complete corrected solver in ```python ... ```.
```

Accuracy Refinement Prompt. Used when the solver is physically valid ($\sigma \geq 0.7$) but nRMSE remains above the early-stopping target ε .

Accuracy Refinement Prompt

```
## Accuracy Refinement -- {pde_name} (current nRMSE = {score})

The solver is functionally correct but not accurate enough.
Improve it to reduce the nRMSE while preserving stability and physical
constraints.

## PDE Description
{pde_description}

{physics_requirements}

## Current Solver
```python
{code}
```

{current_physics_report}

## Improvement Strategies (try the most impactful first)
1. Higher-order spatial discretisation: 2nd/3rd-order Lax-Wendroff, WENO-3/5, or
  compact finite differences.
2. Higher-order time integration: replace forward Euler with RK4 or SSP-RK3;
  use IMEX for stiff problems.
```

3. Spectral methods: FFT-based pseudo-spectral with dealiasing (2/3 rule) for periodic domains.
4. Adaptive sub-stepping: use a smaller internal dt; store only the requested output times.
5. Reduce numerical diffusion: switch to a less diffusive limiter or higher-order reconstruction.

Return the complete improved solver in ````python ... ````.